

High Class Shop — Checklist de Go-Live

Infraestrutura e integrações externas a configurar para colocar a plataforma em produção real · 20/07/2026

Hoje a plataforma roda com integrações em modo sandbox/teste e o banco de dados é tratado como ambiente de demonstração. Este documento lista, integração por integração, o que precisa ser **reconfigurado ou promovido para produção** antes do go-live — não é apenas trocar variáveis de ambiente por deploy.

1. Infraestrutura de deploy

Camada	Serviço	Observação
Backend (API)	Railway	Build via Dockerfile. Root Directory do serviço = <code>backend</code> . Healthcheck: <code>/api/health</code> .
Frontend	Vercel	SPA (rewrite para <code>index.html</code>).
Banco de dados	PostgreSQL (Prisma)	Decisão pendente — ver seção 2.

⚠ O Vercel também mostra um segundo deploy chamado "backend" nos checks de PR — **isso não é um deploy real, é um projeto Vercel não usado**. O único backend real é o do Railway.

⚠ **As migrations do banco não rodam automaticamente no deploy.** O Dockerfile só faz `prisma generate` + `build`. É preciso rodar manualmente `prisma migrate deploy` (banco novo) ou `prisma db push` (banco existente com histórico divergente) antes/junto de cada deploy que alterar o schema.

2. Decisão necessária antes de tudo: banco de produção

O banco atual (Supabase) é tratado internamente como **ambiente de demonstração**, com dados de seed e um histórico de migrations divergente do schema real. Antes do go-live, o time precisa decidir:

- **Opção A** — promover o Supabase atual a produção (risco: dados de demo misturados com dados reais).
- **Opção B (recomendada)** — provisionar um banco novo e limpo só para produção, aplicando `prisma migrate deploy` desde o início (evita o problema de divergência de schema que existe hoje).

3. Integrações externas — o que precisa mudar para produção

Integração	Função na plataforma	Variáveis de ambiente	Ação para ir a produção
DocuSign	Assinatura eletrônica de contratos	<code>DOCUSIGN_INTEGRATION_KEY</code> , <code>DOCUSIGN_USER_ID</code> , <code>DOCUSIGN_ACCOUNT_ID</code> , <code>DOCUSIGN_PRIVATE_KEY</code> , <code>DOCUSIGN_ENV</code> , <code>DOCUSIGN_TEMPLATE_ID</code> , <code>DOCUSIGN_WEBHOOK_SECRET</code> , <code>BACKEND_URL</code>	Conta de produção é separada da sandbox. Solicitar "Go-Live" no painel DocuSign → gera novo Integration Key, novas chaves RSA e novo Template ID. <code>BACKEND_URL</code> = domínio público real.
AWS S3 (ou R2)	Armazenamento de imagens de produto	<code>AWS_REGION</code> , <code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code> , <code>AWS_BUCKET_NAME</code> , opcional <code>AWS_ENDPOINT</code>	Criar bucket dedicado de produção e usuário IAM com permissão mínima (apenas esse bucket).
AWS SES	E-mails transacionais/notificações	<code>EMAIL_FROM</code> + credenciais AWS acima	SES nasce em modo sandbox (só envia para e-mails verificados). Verificar domínio (SPF/DKIM) e solicitar saída do sandbox no console AWS — sem isso, notificações falham em produção.
Calendly	Agendamento de compromissos	<code>CALENDLY_OAUTH_CLIENT_ID</code> , <code>CALENDLY_OAUTH_CLIENT_SECRET</code> , <code>CALENDLY_OAUTH_REDIRECT_URI</code> , <code>CALENDLY_TOKEN_ENCRYPTION_KEY</code> , <code>CALENDLY_WEBHOOK_CALLBACK_URL</code> , <code>CALENDLY_WEBHOOK_SIGNING_KEY</code>	Atualizar redirect URI do app OAuth para o domínio real. Webhook callback precisa ser HTTPS público. Gerar nova chave de criptografia (não reaproveitar a de dev).
Google Meet	Criação de salas de videoconferência	<code>GOOGLE_OAUTH_CLIENT_ID</code> , <code>GOOGLE_OAUTH_CLIENT_SECRET</code> , <code>GOOGLE_OAUTH_REDIRECT_URI</code> , <code>GOOGLE_MEET_TOKEN_ENCRYPTION_KEY</code> , <code>GOOGLE_MEET_ACCESS_TYPE</code> , <code>MEETING_PROVIDER</code> , <code>JITSI_BASE_URL</code>	Conta Google Workspace (não Gmail comum) conectada manualmente pelo admin após o deploy. Tela de consentimento OAuth precisa estar "Published" — em modo teste o token expira em 7 dias.
Google Drive	Importação em lote de imagens	<code>GOOGLE_DRIVE_API_KEY</code>	Sem promoção especial. Restringir a key por domínio/IP se possível.

JWT / Auth	Autenticação e sessões	<code>JWT_SECRET_ACCESS</code> , <code>JWT_SECRET_REFRESH</code> , <code>JWT_SECRET_REFERRAL</code> , <code>JWT_SECRET_ADVISOR</code> , <code>JWT_SECRET_PASSWORD_RESET</code>	Gerar 5 valores novos e aleatórios (ex. <code>openssl rand -base64 48</code>). Nunca reaproveitar os de dev/demo.
CORS / Frontend	Comunicação frontend ↔ backend	<code>FRONTEND_URL</code> (backend), <code>VITE_API_BASE_URL</code> (frontend)	Apontar para os domínios reais de produção (Vercel e Railway).

4. Ordem de execução recomendada

1. Decidir e provisionar o banco de produção (seção 2).
2. Gerar todos os secrets e chaves de criptografia novos.
3. Configurar o lado de produção de cada integração (DocuSign go-live, SES fora do sandbox, redirect URIs de Calendly/Google, consent screen do Google publicado).
4. Criar o bucket S3/R2 de produção.
5. Configurar todas as variáveis de ambiente no Railway (backend) e Vercel (frontend).
6. Rodar `prisma migrate deploy` (banco novo) ou `prisma db push` (banco existente) manualmente.
7. Deploy do backend (Railway) e do frontend (Vercel).
8. Um admin reconecta a conta Google Meet pelo painel (`/admin/settings`) já no ambiente de produção.
9. Validar com `GET /api/health/detailed` — deve retornar `hasDatabase`, `hasAwsConfig`, `hasJwtSecrets` e `hasDocuSign` como `true`.
10. Teste ponta a ponta: processo → agendamento (Calendly) → reunião (Meet) → proposta → contrato assinado (DocuSign) → e-mail de notificação (SES).